Progress Report

Ayush Madhav Kumar

RL for Jac Code Generation — July 2026

UNIVERSITY OF MICHIGAN

JASECI

Goal & experimental setup

- Goal: a local model that writes correct Jac, compiles and prints the exact expected output.
- Hardware: Apple M5 Pro, 48 GB unified memory, MLX LoRA. Trains the 30B-A3B MoE ($\sim$3B active); every dense $\geq$14B and larger MoE OOMs.
- Models: fresh `Qwen3-Coder-30B-A3B` vs. `jac-qwen3coder` (same base, already SFT+DPO'd on Jac).
- Grader: Jac compiler + runtime, exact-stdout = pass. No learned reward model anywhere.

Task families

- Hole-fill: 84 deterministic tasks mined from `this_is_jac`
- Conversion: python$\rightarrow$jac
- Graph walkers: node/edge/walker (OSP) idiom
- Synthetic: 32 fresh deterministic tasks

Three eras of experiments

Era 1: weekend GRPO

Jun 20–21

GRPO on both models, STaR loop.

“Supervised levers move models; RL does not.”

Era 2: SFT ladder

Jun 25–28

Leak-free harness, 30-cell ladder, tuned-GRPO arm.

“RL is a dead end.” (v1)

Era 3: the correction

Jul 1–2

Eval *and* reward shared a broken extractor.

Everything re-measured.

Every headline number from Eras 1–2 was measured wrong: a $\sim$3.5–4$\times$ undercount.

UNIVERSITY OF MICHIGAN

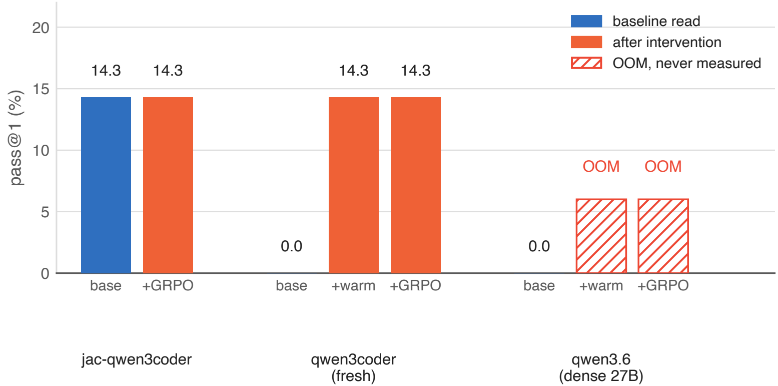
Era 1: weekend GRPO, everything at 14.3%

1. Metal OOM: group 6 / completion 512 blew 48 GB at iter 2; survived at group 4 / comp 256 (∼38 GB peak).
2. $\sigma = 0$ trap: at 0% base pass every rollout scores the same, so advantage $(r - \bar{r})/\sigma = 0$, KL $= 0.0$ forever. RL cannot bootstrap a skill the base has none of.
3. Splice bug: models emit the whole `can ... {}` unit; the hole sat *inside* it, so the result was nested broken Jac. Base, warm (SFT loss 0.006!), and GRPO scored identically: the splice was broken, not the models.
4. Dense wall: Qwen3.6-27B SFT OOM'd at iter 1 in every config; the whole line is inference-only on 48 GB.
5. LoRA-GRPO null: splice fixed, real reward variance (σ up to 0.11), and +GRPO output was still byte-identical to its start point, KL $\approx$ 0.

Phase 1 (31 tasks, holdout 7)	pass
jac-qwen3coder base	14.3%
+ GRPO	14.3%
qwen3coder base	0%
+ warm-start SFT	14.3%
+ GRPO	14.3%

Phase 2.5 (51 tasks, holdout 12): pass@1 $=$ pass@8 $=$ 14.3% for every arm. STaR flickered to pass@4 $=$ 25% for one round, then died.

Era 1, in one chart: four interventions, one number



The dense 27B never produced a number, it OOM'd before either intervention finished. Of the two models that did run: four interventions, three identical readings. A tell, in hindsight, that the splice bug (#3) was capping the score, not the models.

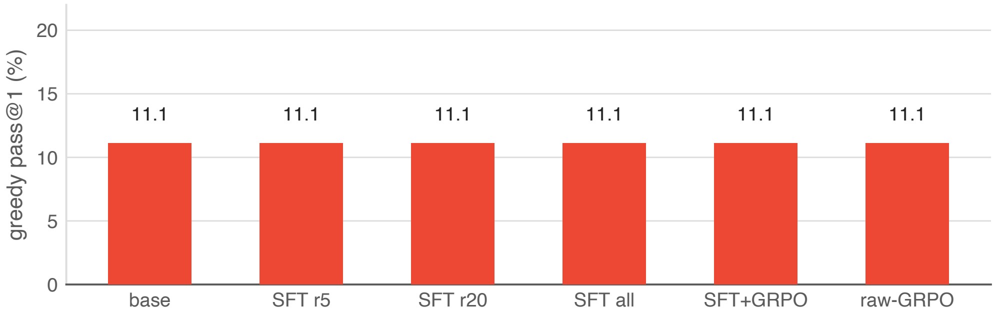
Era 2: the SFT ladder, built to settle it

- Design (leak-free rebuild): rungs train-$N \in \{1, 3, 5, 10, 20, \text{all}\}$; conditions base, SFT, SFT+GRPO, raw-GRPO control, tuned-GRPO (500 iters, $10\times$ LR).
- 2 models $\times$ 30 cells; corpus grown 51$\rightarrow$84 deterministic tasks.
- Sound train/holdout splits, monotone reward, Wilson CIs throughout, built to be trustworthy this time.

Why a flat result is a strong prior something's wrong

A real signal usually varies at least a little between conditions. A number that's *exactly* flat across 30 cells of real training runs says you're not measuring the model. You're measuring a fixed artifact of the harness. That's what the next slide shows, and at the time it read as proof.

Era 2, in one chart: the same shape, wrong story



Every condition pinned at 11.1%: the Jac specialist frozen at literally the same number whether trained on 5 tasks or all 84, with or without GRPO, tuned or not. (Compare this shape to the corrected version two slides on.)

v1 verdict (Jun 28): "RL is a dead end," declared done. All of it rested on this chart.

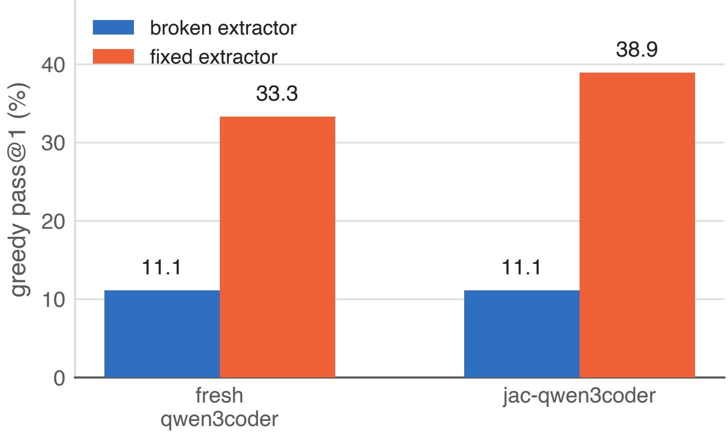
The measurement bug that faked the dead end

- `extract_jac`/`unwrap_unit` grabbed the driver docstring instead of the model's answer whenever the model echoed the whole driver, so it auto-failed.
- The GRPO reward used the same extractor, so GRPO was trained *and* scored on garbage. Era 1's real LoRA-GRPO null and this eval bug were two separate problems stacked on top of each other.
- Fixed in `8164ee2`: brace-matched, name-targeted body extraction, then everything re-run.

Lesson

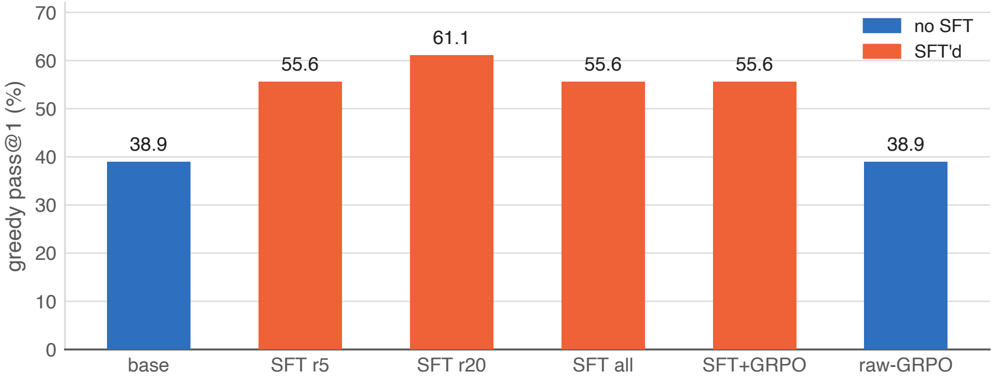
Verify the grader before trusting a null result. One shared helper silently capped three weeks of experiments.

Era 3, in one chart: the ∼3.5× undercount



Same models, same holdout ($n=18$): about a ∼3.5× undercount, uniformly. Everything from this point on is measured with the fixed extractor.

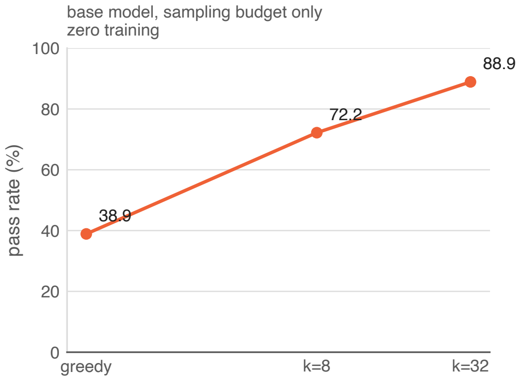
Corrected ladder: SFT works (pure-fn holdout, $n = 18$)



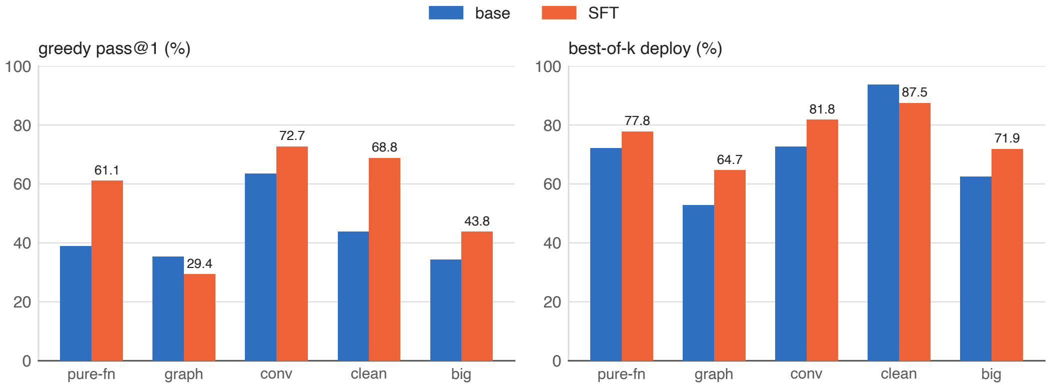
SFT lifts greedy 38.9 to 61.1% (+22pp, sweet spot at rung-20; rung-all drops to 55.6, one task regressed from task interference). GRPO is about equal to SFT (55.6 either way, a *valid* null now). raw-GRPO from base equals base: GRPO can't bootstrap. Best-of-k deploy: 72.2 to 77.8%.

The gap is syntax, and the compiler closes it for free

- jac-qwen3coder: pass@1 38.9% vs pass@8 72.2%, a +33.3pp gap. The right Jac is *reachable*; greedy lands on a near-valid variant that won't compile.
- Failures are compile-fails, not wrong answers (missing ';', `here.jid` vs `jid(here)`).
- Fresh model: pass@1 $=$ pass@8 $=$ 33.3%, gap 0, a real dead end.
- The Jac compiler is a free, perfect verifier: sample k, keep the first that compiles. Deploy $=$ oracle throughout.



Across all five holdouts



Conversion is the peak: 72.7% greedy, 81.8% best-of-k. Richer spec beats hole-fill by +9–11pp.
Graph walkers: greedy dips under SFT (noise), but best-of-k still gains, 52.9 to 64.7%.

n: pure-fn 18 · graph 17 · conversion 11 · clean 16 · big+fresh 32

Follow-ups: ceiling, honest gap, generalization

E1 · true ceiling $\sim$94%

Drop 2 junk regex tasks ($n = 16$): base best-of-k 93.8%. SFT sharpens greedy (43.8 $\rightarrow$ 68.8%) but *narrows* sampling: its best-of-k (87.5%) dips one task below base.

Exploit vs explore: single-shot $\rightarrow$ SFT; k-budget $\rightarrow$ base + best-of-k.

E2 · the honest gap

Free-form NL $\rightarrow$ Jac: 0/3 for both the hole-fill-SFT and conversion-SFT models.

Not fixed by model choice, needs its own dataset. The one real remaining weakness.

E3 · SFT generalizes

Bigger, fresher holdout ($n = 32$, +16 synthetic): greedy 34.4 $\rightarrow$ 43.8%, best-of-k 62.5 $\rightarrow$ 71.9%.

Lift holds on tasks that didn't exist at training time (synthetic 4 $\rightarrow$ 5, library 7 $\rightarrow$ 9), not memorization.

Why GRPO $\approx$ SFT, and why that's a real result

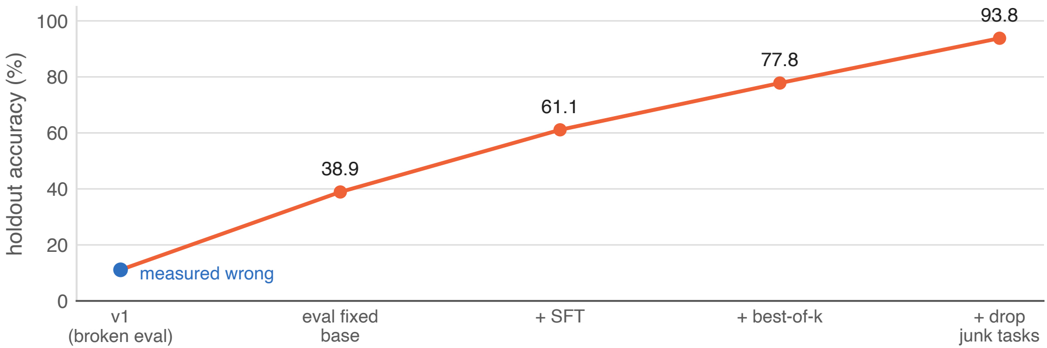
- Every measurement here agrees: GRPO on top of SFT adds nothing beyond SFT alone; raw GRPO from the base moves nothing. This is Era 1's LoRA-GRPO null (KL$\approx$0, byte-identical output) showing up again at scale.
- Matches Yue et al. 2504.13837: RL sharpens a model's *sampling distribution* around capability it already has. It doesn't expand the underlying boundary.
- Once SFT already moves greedy close to the model's own pass@k ceiling, there's little room left to sharpen further, and LoRA's limited capacity (same constraint behind task interference) leaves even less room to search past it.

What would actually expand it

- Full fine-tune: removes LoRA's capacity ceiling entirely. Untested here; needs $>$48 GB.
- Distillation / expert iteration: train on a stronger teacher's, or the model's own compiler-verified best, outputs. Injects capability from *outside* rather than re-weighting what's already there.

UNIVERSITY OF MICHIGAN

The journey: a bug hid an 11 $\rightarrow$ 94% generator



- Verify the grader before trusting a null: eval *and* reward shared the bug.
- The gap is syntax, not capability: compiled Jac is almost always exactly right.
- Compiler equals a free, perfect verifier: best-of-k ships 78–94% with zero training.
- SFT works and generalizes; sweet spot $\sim$20 tasks, more causes interference.
- LoRA-GRPO can't bootstrap and lands $\approx$ SFT after warm-start (matches Yue et al. 2504.13837).
- Conversion beats hole-fill by +9–11pp: richer, unambiguous spec.

Shipped

- `rl/generate.py`: best-of-k generator, sample k at $T{=}0.8$, return the first candidate the Jac compiler accepts. Live-verified; deploy $=$ oracle.
- Studio GENERATE panel, plus all corrected result charts.
- Studio RL data pipeline: SFT / GRPO / conversion / synthetic datasets and splits, browsable end-to-end.
- Broken-eval numbers flagged with banners across all docs.

Deployable today, no further training

pure functions (best-of-k)	$\sim$78%
conversion + SFT	82%
clean tasks	94%
at $k = 32$	89%

Next steps

- Close the E2 gap: SFT on free-form NL $\rightarrow$ Jac prompts, the one measured weakness.
- Spec $\rightarrow$ Jac dataset for the graph-walker idiom: conversion can't source it (no Python equivalent of OSP).
- Curriculum / quality-filter the train pool: kill the rung-all task interference (or keep training at $\sim$20).
- Distillation / expert iteration: the one method that can expand the capability boundary, not just sharpen it.
- Whole-file generation track: bridge from hole-fill to generating full Jac programs; gets its own ladder.
- Guardrails: partial-credit grading; Wilson CIs on every claim. "Gap closed" means SFT pass@1 CI reaches base pass@8.

Fixing the graph-walker weak spot

Graph holdout ($n = 17$)

	base	+SFT
greedy pass@1	35.3%	29.4%
best-of-k	52.9%	64.7%

Only family where SFT hurts greedy: the tell for task interference. The SFT mix is dominated by hole-fill/conversion, graph examples get outvoted.

Fix plan, priority order

1. Dedicated graph-walker dataset: spec→jac synthetic set for the OSP idiom (`walker/node/edge/visit`). Conversion structurally can't source this, Python has no walker equivalent.
2. Curriculum-weighted SFT / per-family adapter: fix the interference itself, not just add data; graph examples get proportional gradient signal instead of being outvoted by volume.
3. Stopgap, zero training, ships today: detect task type in `rl/generate.py`, bump $k \to 32$ specifically for graph-walker prompts.

#1 and #2 are the actual fix. #3 is a band-aid you can flip on right now.

Glossary: terms & context

- Jac: Jaseci's language; supersets Python, adds graph-native OSP constructs.
- OSP / walkers: `node`/`edge`/`walker` idiom; no Python equivalent.
- SFT: supervised fine-tuning on prompt$\longrightarrow$answer pairs.
- DPO: preference optimization on chosen-vs-rejected pairs.
- GRPO: RL, sample rollout groups, advantage $= (r - \bar{r})/\sigma$.
- tuned-GRPO: GRPO variant, 500 iters, 10$\times$ LR; rules out "just needs more training."
- LoRA: low-rank adapters; only small matrices train.
- MoE / 30B-A3B: 30B params, $\sim$3B active/token.
- MLX: Apple's on-device ML framework; runs the LoRA training here.
- Metal: Apple's GPU compute API; where the OOMs actually happen.
- unified memory: Apple silicon's shared CPU/GPU RAM; 48 GB hard ceiling here.
- warm-start: brief SFT before RL so rollouts aren't all-fail.
- STaR: sample, keep verified wins, SFT on them, repeat.
- $\sigma = 0$ trap: equal rollout rewards, so zero gradient; can't bootstrap from 0% skill.
- KL: divergence from start policy; $\approx$0 means the model didn't move.
- this_is_jac: real Jac codebase mined for hole-fill tasks; ground-truth source.
- exact-stdout: grading rule, output must byte-match; no partial credit.
- Studio: this repo's app; ships the GENERATE panel + RL data pipeline live.
- holdout: eval tasks never trained on; five sets (n=18/17/11/16/32).
- hole-fill: one function body blanked from a working Jac file.
- conversion: python$\longrightarrow$jac translation; richest spec, best scores.
- ladder / rung-N: train on N tasks, eval on holdout.
- task interference: harder train tasks regress ones already learned.
- greedy / pass@1: one deterministic (temp-0) answer; the headline metric.
- pass@k (oracle): any of k samples correct; ceiling if perfectly picked.
- best-of-k (deploy): sample k, return first the compiler accepts.
- Wilson CI: binomial confidence interval; the noise bar on every rate.
- expert iteration / distillation: train on a stronger model's verified outputs.